

Variables & Data Types

ELEC1006: ENGINEERING COMPUTING

Variable

- A variable holds a specific type of data.
- To create a variable, you must write a variable definition (also called variable declaration).
 - This is not required for MATLAB.

Variable Definitions

- Variable definitions specifies the variable name and the type of data a variable holds.
- Example:

```
int hour;          /* The word int specifies the variable  
                   hour is 4 bytes long. */
```

```
double rate, pay;  /* The word double specifies a  
                   double-precision floating-  
                   point numbers. */
```

Variable Naming Conventions

- 1st character of a variable name must be either an alphabet or underscore (_).
 - No numbers allowed: ~~X~~ `int` 2hour; ~~X~~
- After the 1st character, characters can be any combination of alphabets, numbers and underscore: `int` hour_2;
- Case sensitive: Variables denoted by upper and lower case alphabets are distinct, e.g. `hour` is a separate variable from `Hour`.

Data Types

Name	Description	Size*	Range*
char	Character or small integer.	1byte	signed: -128 to 127 unsigned: 0 to 255
short int (short)	Short Integer.	2bytes	signed: -32768 to 32767 unsigned: 0 to 65535
int	Integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
long int (long)	Long integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
bool	Boolean value. It can take one of two values: true or false.	1byte	true or false
float	Floating point number.	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	Double precision floating point number.	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	Long double precision floating point number.	8bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	Wide character.	2 or 4 bytes	1 wide character

Source: <https://cplusplus.com>

Data Type Sizes

- 1 byte = 8 bits
- 8 bits can represent $2^8 = \underline{256}$ numbers:
 - a) 0 to 255 (**unsigned int**: only positive integers).
 - b) -128 to 127 (**signed int**: negative and positive integers).

Size	Unique representable values	Notes
8-bit	256	$= 2^8$
16-bit	65 536	$= 2^{16}$
32-bit	4 294 967 296	$= 2^{32}$ (~4 billion)
64-bit	18 446 744 073 709 551 616	$= 2^{64}$ (~18 billion billion)

Variable Definition

- Variables can be defined
 - In separate lines:

```
int length;  
int width;  
unsigned int area;
```
 - In the same line:

```
int length, width;  
unsigned int area;
```

Variable Assignment

- char data type:

```
char letter = 'C';
```

- float data type:

```
float x1 = 3.14159;
```

- double data type:

```
double x2 = 6.25e-5;
```

Note: $6.25e-5 = 6.25 \times 10^{-5}$

- bool data type:

```
bool a = true;    % true = 1
```

```
bool b = false;   % false = 0
```


Variable Initialisation

- An assignment statement uses the **=** operator to store a value in a variable.
`int item;`
`item = 12;      // Acceptable`
- But this type of assignment will not work:
`12 = item;      // Unacceptable`

Scope of a Variable

- The scope of a variable: the part of the program in which the variable can be accessed.
- A variable cannot be used before it is defined.

```
#include <iostream>

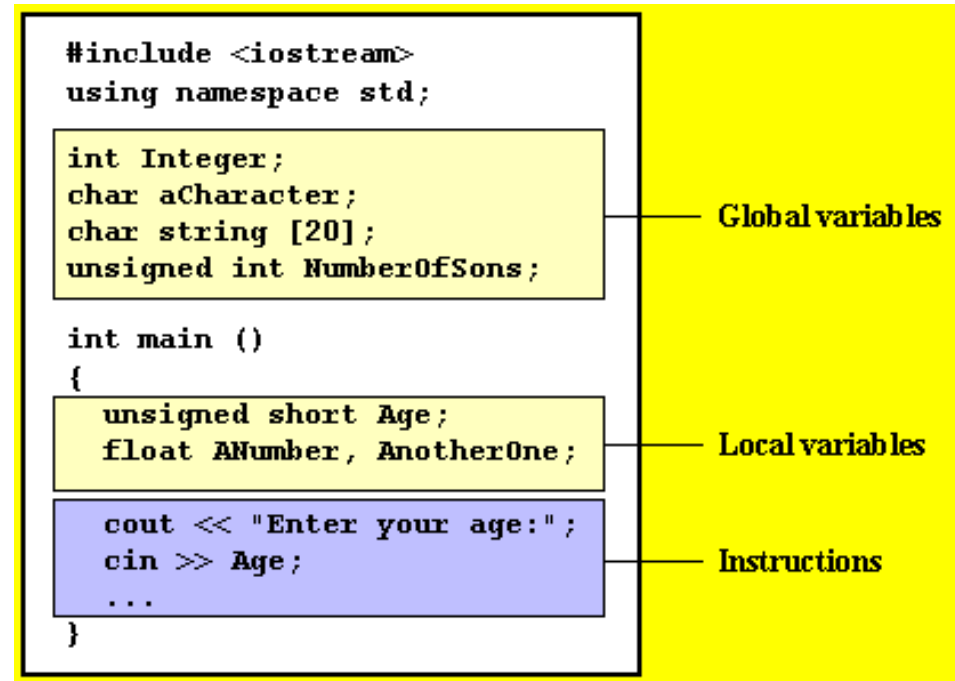
using namespace std;

int main()
{
    cout<<"The income of the company is: "<< income;
    double income;
    return (0);
}
```

Error – this program will not compile as income is used before being declared.

Global & Local Variables

- A **global variable** is a variable declared in the main body of the source code, outside all functions. It can be referred from anywhere in the code.
- A **local variable** is one declared within the body of a function or a block - scope of local variables is limited to the block enclosed in braces ({}), where they are declared



From cplusplus.com

Named Constant

- Named constant (constant variable): variable whose content cannot be changed during program execution
- Used for representing constant values with descriptive names:

```
const double TAX_RATE = 0.0675;
```

```
const int NUM_STATES = 50;
```

```
TAX_RATE = 0.07; // this will result in an  
error in compiling
```

- Often named in uppercase letters

const VS. #define

- `#define` – C-style of naming constants:
 `#define NUM_STATES 50`
 – Note no semicolon at end
- Interpreted by pre-processor rather than compiler
- Does not occupy memory location like `const`

More info on C++

- [1] cplusplus.com: Variables and types
<https://cplusplus.com/doc/tutorial/variables/>